

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

**Отчёт по лабораторным работам №1 и №2
по курсу «Операционные системы и архитектура компьютера»**

Выполнил: В. Е. Авраменко

Группа: М8О-204БВ-25

Преподаватель: Н. А. Гапонов

Дата: 08.10.2026

Оценка: _____

Москва, 2026

Цель и условия работ

Цель сквозной лабораторной работы №1 – получить профиль системных вызовов работающей программы, сгруппировать вызовы по назначению и сопоставить их с наблюдаемым поведением. В качестве объекта исследования выбрана GNU `ls`. В лабораторной работе №2 реализована передача текстовых команд от родительского процесса дочерней программе через неименованный канал и запись вычисленных сумм в файл.

Выполнена группа 1, вариант 1 лабораторной работы №2. Трассировки получены 8 октября 2026 года в Linux `x86_64` под WSL2: ядро 6.6.87.2-microsoft-standard-WSL2, GCC 13.3.0, `strace` 6.8, GNU `coreutils` 9.4.

Этот документ является промежуточным отчётом по ЛР №1 и №2. Согласно заданию ЛР №1, итоговую сводную аналитику системных вызовов следует дополнить после выполнения остальных лабораторных работ курса.

Лабораторная работа №1: профиль программы `ls`

Сценарий и результат

Для наблюдения за системными вызовами выполнена команда из корня проекта:

```
| LC_ALL=C strace -o report/inc/ls-trace.txt ls -l report/inc/fixture >  
| ↪ report/inc/ls-output.txt
```

Каталог содержит файл `alpha.txt` и вложенный каталог `nested`. Ключ `-l` запрашивает подробный список с метаданными. Обычные данные из файлов команда не читает. Программа завершилась с кодом 0; её вывод сохранён в `report/inc/ls-output.txt`, полная трасса – в `report/inc/ls-trace.txt`.

В сохранённой трассе насчитано 123 системных вызова. Это число обращений в данном запуске, а не время выполнения. Вызовы сгруппированы по их наблюдаемому назначению.

Таблица 1 – Группы системных вызовов GNU ls

Группа	Вызовы и число	Назначение
Запуск и завершение	execve (1), exit_group (1)	Запуск программы и завершение с кодом 0.
Файлы и библиотеки	openat (11), read (14), pread64 (2), close (17), lseek (3), access (2)	Загрузка библиотек и чтение конфигурации, баз пользователей, групп и часового пояса.
Каталог и метаданные	getdents64 (2), statx (3), fstat (14), newfstatat (3), statfs (2), lgetxattr (3), listxattr (3)	Перечисление каталога, получение свойств файлов и проверка атрибутов файловой системы.
Память	brk (3), mmap (17), mprotect (5), munmap (1)	Выделение памяти процесса, отображение библиотек и изменение защиты страниц.
Вывод	write (1), ioctl (1)	Запись списка в stdout и проверка, является ли stdout терминалом.
Служба имён	socket (4), connect (4)	Попытки обратиться к локальному сокету nsd; затем читаются системные базы имён.
Среда выполнения	arch_prctl, set_tid_address, set_robust_list, rseq, prlimit64, getrandom (по 1)	Инициализация состояния потока и служебных данных среды выполнения.

В трассе `openat` открывает исследуемый каталог, а `getdents64` возвращает записи каталога; следующий вызов `getdents64` возвращает 0, показывая конец списка. Вызовы `statx` получают метаданные элементов для подробного формата. Для вывода имён владельцев и групп `ls` обращается к NSS и читает системные базы. Итоговая строка записывается через `write` в файловый дескриптор 1.

Ожидаемые нефатальные ошибки: `ENOENT` при проверке необязательного файла и сокета `nsd`. Запрос отсутствующей метки SELinux дал `ENODATA`, а проверка перенаправленного `stdout` – `ENOTTY`. Программа всё равно вывела список и завершилась успешно.

Лабораторная работа №2: вариант 1, группа 1

Задание и метод решения

Программа состоит из двух процессов. Родитель читает имя выходного файла, создаёт канал и порождает дочерний процесс. Дочерний процесс перенаправляет свой стандартный ввод на конец чтения канала и запускает отдельную программу `child`. Родитель передаёт строки команд через конец записи. Дочерняя программа разбирает числа в строке, вычисляет сумму и записывает её в файл. Пустая строка или конец ввода завершают передачу.

Для этого варианта достаточно одностороннего канала: команды идут от родителя к ребёнку, а обратный канал не используется. После `fork` процессы закрывают неиспользуемые концы канала. Родитель закрывает канал записи после ввода и вызывает `waitpid`; закрытие всех дескрипторов записи позволяет ребёнку получить конец файла при чтении.

Числа проверяются на диапазон `int`; сумма накапливается в `long` с проверкой переполнения. Некорректная строка диагностируется, не записывается, а обработка следующих строк продолжается. Ошибка приводит к ненулевому статусу дочерней программы. Пустые строки и строки без чисел не создают запись результата.

Файлы проекта и основные функции

Таблица 2 – Структура исходного кода

Файл	Назначение
<code>parent.c</code>	Настройка SIGPIPE, создание канала и ребёнка, отправка строк, ожидание завершения.
<code>parent_input.c</code> <code>parent_input.h</code>	Динамическое чтение имени файла и командных строк родителем.
<code>child.c</code>	Открытие выходного файла, чтение строк из <code>stdin</code> , запись сумм и обработка ошибок.
<code>sum.c/.h</code>	Разбор строки, проверка токенов и диапазонов, расчёт суммы.
<code>pipe_io.c/.h</code>	Полная запись буфера и динамическое чтение строки из файлового дескриптора.
<code>status.h</code>	Общий перечислимый тип статусов обработки.
<code>Makefile</code>	Сборка исполняемых файлов <code>parent</code> и <code>child</code> .

Функция `run_pipeline` подготавливает сигнал, вызывает `pipe` и `fork`. В дочерней ветке `start_child` выполняет `dup2` и `execv`. В родительской ветке `run_parent` закрывает лишний дескриптор, вызывает `send_commands`, закрывает канал и дожидается ребёнка. В программе ребёнка `process_lines` читает команды и вызывает `sum_line`; `write_sum` передаёт отформатированный результат через `write_all`.

Основные типы данных: `pid_t` хранит идентификатор дочернего процесса, массив `int pipe1[2]` — дескрипторы чтения и записи канала, а `char *` вместе с `size_t` используется для строк произвольной длины. Тип `ssize_t` позволяет вернуть число прочитанных байтов либо `-1` при ошибке; перечисление `Status` различает успешную обработку, конец ввода и ошибки разбора или ввода-вывода.

Таблица 3 – Основные функции и их интерфейсы

Функция	Параметры, результат и назначение
<code>read_filename(char **out)</code>	Возвращает <code>Status</code> ; выделяет строку с именем файла для дочерней программы.
<code>read_command_line(char **out, size_t *out_len)</code>	Возвращает <code>Status</code> ; читает команду родителя и сообщает её длину.
<code>write_all(int fd, const char *buf, size_t len)</code>	Возвращает <code>0</code> или <code>-1</code> ; записывает весь буфер, повторяя запись после <code>EINTR</code> .
<code>read_line(int fd, char **line, size_t *cap)</code>	Возвращает число прочитанных байтов или <code>-1</code> ; динамически читает строку из дескриптора.
<code>sum_line(const char *line, Status *status)</code>	Возвращает сумму типа <code>long</code> , а через <code>status</code> сообщает об ошибке токена или переполнении.
<code>process_lines(int fd)</code>	Читает команды дочернего процесса, вызывает <code>sum_line</code> и записывает успешные суммы; возвращает код завершения.

Системные вызовы

Таблица 4 – Системные вызовы, наблюдавшиеся при запуске ЛР №2

Вызов ядра (число)	Назначение в этом запуске
rt_sigaction (1)	Настроить игнорирование SIGPIPE, чтобы родитель мог обработать EPIPE.
pipe2 (1)	Создать pipe; это реализация библиотечного вызова pipe в данной среде.
clone (1)	Создать дочерний процесс; в этой среде вызван библиотечной функцией fork.
dup2 (1)	Подключить конец чтения канала к стандартному вводу ребёнка.
execve (2)	Запустить parent под трассировщиком и заменить образ ребёнка программой child; вызов ребёнка сделан через execv.
openat (5)	Открыть выходной файл ребёнком и файлы динамического загрузчика.
read (40)	Прочитать пользовательский ввод, команды из pipe и данные библиотек.
write (7)	Передать команды по pipe, записать суммы и вывести сообщение о завершении.
close (9)	Закрыть концы pipe, выходной файл и дескрипторы загрузчика.
wait4 (1)	Дождаться ребёнка; это реализация библиотечного вызова waitpid.
exit_group (2)	Завершить родительский и дочерний процессы.

Таблица 5 – Вызовы динамического загрузчика и среды выполнения в той же трассе

Вызов ядра (число)	Наблюдаемое назначение
access (2), openat и read (учтены выше)	Проверка необязательного файла загрузчика и чтение файлов библиотек и конфигурации.
arch_prctl (2), set_tid_address (2)	Настройка thread-local storage и служебного адреса каждого процесса.
brk (6), mmap (16), munmap (2)	Управление кучей и отображениями исполняемых файлов, библиотек и кэша загрузчика.
fstat (6), pread64 (4)	Получение метаданных и чтение заголовков ELF-файла библиотеки.
getrandom (2)	Получение случайных данных, запрошенных средой выполнения.
mprotect (6)	Установка прав доступа к сегментам памяти исполняемых файлов и библиотек.
prlimit64 (2)	Чтение ограничения размера стека при запуске процессов.
rseq (2), set_robust_list (3)	Регистрация служебных структур потоков в среде выполнения.

В обеих таблицах приведены все 24 различных имени системных вызовов из сохранённой трассы; повторные вызовы отражены в счётчиках. Часть обращений к openat, read, close и другим функциям относится к динамическому загрузчику и libc, поэтому она отделена от действий прикладного кода. Имена clone, pipe2, execve и wait4 зависят от реализации libc и ядра.

Ключевые фрагменты кода

Родитель создаёт канал и дочерний процесс; ошибки системных вызовов проверяются до продолжения работы.

Листинг 1 – Создание канала и дочернего процесса (parent.c)

```
static int run_pipeline(char *filename) {
    if (ignore_sigpipe() != EXIT_SUCCESS) {
        return EXIT_FAILURE;
    }
    int pipe1[2];
    if (pipe(pipe1) == -1) {
        perror("pipe");
        return EXIT_FAILURE;
    }
    if (fflush(stdout) == EOF) {
        perror("fflush");
        close_descriptor(pipe1[0]);
        close_descriptor(pipe1[1]);
        return EXIT_FAILURE;
    }
    pid_t pid = fork();
    if (pid == 0) {
        start_child(pipe1, filename);
    }
    if (pid == -1) {
        perror("fork");
        close_descriptor(pipe1[0]);
        close_descriptor(pipe1[1]);
        return EXIT_FAILURE;
    }
    return run_parent(pipe1, pid);
}
```

Дочерняя ветка подключает чтение канала к стандартному вводу и запускает отдельную программу.

Листинг 2 – Перенаправление stdin и запуск child (parent.c)

```
static void start_child(int pipe1[2], char *filename) {
    if (dup2(pipe1[0], STDIN_FILENO) == -1) {
        perror("dup2");
        _exit(EXIT_FAILURE);
    }
    if (pipe1[0] != STDIN_FILENO && close_descriptor(pipe1[0]) != EXIT_SUCCESS) {
        _exit(EXIT_FAILURE);
    }
    if (close_descriptor(pipe1[1]) != EXIT_SUCCESS) {
        _exit(EXIT_FAILURE);
    }
    execv("./child", (char *[]){"./child", filename, NULL});
    perror("execv");
    _exit(EXIT_FAILURE);
}
```

Родитель отправляет строки, закрывает канал, чтобы передать конец ввода, и дожидается кода завершения ребёнка.

Листинг 3 – Передача команд и ожидание ребёнка (parent.c)

```
static int send_commands(int fd) {
    for (;;) {
        char *line = NULL;
        size_t len = 0;
        Status status = read_command_line(&line, &len);
        if (status == STATUS_END_OF_INPUT) {
            return EXIT_SUCCESS;
        }
        if (status != STATUS_OK) {
            fprintf(stderr, "parent: ошибка чтения команды\n");
            return EXIT_FAILURE;
        }
        int result = write_all(fd, line, len);
        if (result == -1) {
            perror("write pipe1");
        }
        free(line);
        if (result == -1) {
            return EXIT_FAILURE;
        }
    }
}

static int run_parent(int pipe1[2], pid_t pid) {
    int result = close_descriptor(pipe1[0]);
    if (result == EXIT_SUCCESS) {
        result = send_commands(pipe1[1]);
    }
    if (close_descriptor(pipe1[1]) != EXIT_SUCCESS) {
        result = EXIT_FAILURE;
    }
    if (wait_for_child(pid) != EXIT_SUCCESS) {
        result = EXIT_FAILURE;
    }
    return result;
}
```

Дочерний процесс проверяет числа и записывает сумму только для корректной строки.

Листинг 4 – Проверка целых чисел и накопление суммы (sum.c)

```
long sum_line(const char *line, Status *status) {
    char *copy = strdup(line);
    if (copy == NULL) {
        *status = STATUS_IO_ERROR;
        return 0;
    }

    long sum = 0;
    int had_number = 0;
    *status = STATUS_OK;

    char *saveptr = NULL;
    char *tok = strtok_r(copy, " \\t\\r\\n\\v\\f", &saveptr);
    while (tok != NULL) {
        char *end = NULL;
        errno = 0;
        long value = strtol(tok, &end, 10);
        if (end == tok || *end != '\\0' || errno == ERANGE ||
            value < INT_MIN || value > INT_MAX) {
            const char *reason = end == tok || *end != '\\0'
                ? "не число" : "число вне диапазона int";
            fprintf(stderr, "child: %s: \\\"%s\\\"\\n", reason, tok);
            *status = STATUS_BAD_NUMBER;
        } else if ((value > 0 && sum > LONG_MAX - value) ||
            (value < 0 && sum < LONG_MIN - value)) {
            fprintf(stderr, "child: переполнение суммы\\n");
            *status = STATUS_BAD_NUMBER;
        } else {
            sum += value;
            had_number = 1;
        }
        tok = strtok_r(NULL, " \\t\\r\\n\\v\\f", &saveptr);
    }

    free(copy);
    if (!had_number && *status == STATUS_OK) {
        *status = STATUS_NO_NUMBERS;
    }
    return sum;
}
```

Результаты проверки

В зафиксированном запуске входной файл содержал имя результата и три строки чисел:

Листинг 5 – Входные данные опыта ЛР №2

```
reports/lab2-result.txt
1 2 3
10 -5
-2147483648 2147483647
```

Выходной файл содержал суммы 6, 5 и -1. Родитель сообщил о завершении ребёнка с кодом 0. Для воспроизведения из каталога проекта используются команды:

```
cd src
make
```

```
mkdir -p reports
strace -f -o /tmp/lab2-trace.txt ./parent < ../report/inc/lab2-input.txt >
↪ /tmp/lab2-stdout.txt
cat reports/lab2-result.txt
```

В зафиксированном запуске выходной файл назывался `reports/lab2-result.txt`; приведённая команда воспроизведения создаёт для него каталог. В короткой проверке без трассировки используется файл `result.txt`:

```
cd src
make
printf 'result.txt\n1 2 3\n10 -5\n\n' | ./parent
cat result.txt
```

Ожидаются строки 6 и 5, а родитель завершится с кодом 0. Сборка выполняется с флагами `-std=c11 -Wall -Wextra -Wpedantic -D_POSIX_C_SOURCE=200809L`.

Дополнительно проверены граничные ситуации:

Таблица 6 – Проверка обработки ошибок и формата ввода

Сценарий	Фактический результат
Имя файла и пустая завершающая строка в формате CRLF	Файл создаётся без завершающего символа возврата каретки; сумма 1 2 3 записывается как 6, код завершения 0.
В команде есть число за пределами <code>int</code>	Печатается диагностика; эта строка пропускается, последующие корректные строки обрабатываются, итоговый код равен 1.
Каталог выходного файла отсутствует	Дочерняя программа сообщает об ошибке <code>open</code> , оба процесса завершаются ненулевым кодом без зависания.

Анализ трассы ЛР №2

Трасса содержит 125 вызовов, если считать каждый начатый вызов один раз и не считать строки `resumed` повторно. В ней наблюдаются создание одного канала и одного дочернего процесса, два запуска исполняемых образов, перенаправление `stdin` ребёнка, запись трёх команд и трёх сумм, закрытие канала и ожидание ребёнка.

Листинг 6 – Создание канала и дочернего процесса

```
1419 rt_sigaction(SIGPIPE, {sa_handler=SIG_IGN, ...}, NULL, 8) = 0
1419 pipe2([3, 4], 0) = 0
1419 clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
↪ ...) = 1420
1420 dup2(3, 0 <unfinished ...>
1419 write(4, "1 2 3\n", 6 <unfinished ...>
1420 execve("./child", ["/child", "reports/lab2-result.txt"], ...) <unfinished ...>
```

Запись подтверждает передачу команд по дескриптору 4. В дочернем процессе дескриптор 3 становится стандартным вводом. PID в трассе относится к конкретному запуску и при повторении изменится.

Листинг 7 – Запись результатов и ожидание

```
1420 write(3, "6\n", 2) = 2
1420 write(3, "5\n", 2) = 2
1420 write(3, "-1\n", 3) = 3
1419 wait4(1420, <unfinished ...>
```

Библиотечные функции `write_all` и `read_line` не являются системными вызовами; их обращения к ядру видны как отдельные `write` и `read`. В сохранённом опыте зафиксировано 40 чтений и 7 записей. Побайтовое чтение упрощает обработку строк, но увеличивает число обращений к ядру. Порядок строк с пометками `unfinished` и `resumed` отражает чередование процессов во время трассировки.

Выводы

Для сквозной работы получен профиль системных вызовов GNU `ls` и реализована меж-процессная передача команд через неименованный канал. В трассах наблюдаются операции чтения каталога и метаданных, а также вызовы `pipe2`, `clone`, `dup2`, `execve`, файлового ввода-вывода и ожидания процесса. Результат ЛР №2 подтверждён выходным файлом и нулевым кодом завершения ребёнка. Сохранённые трассы относятся к реальным запускам; системные номера процессов и детали конкретной среды могут изменяться.

Приложение А. Исходный код проекта

Листинг А.1 – Файл parent.c

```
#include <errno.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

#include "parent_input.h"
#include "pipe_io.h"
#include "status.h"

static int close_descriptor(int fd) {
    if (close(fd) == -1) {
        perror("close");
        return EXIT_FAILURE;
    }
    return EXIT_SUCCESS;
}

static void start_child(int pipe1[2], char *filename) {
    if (dup2(pipe1[0], STDIN_FILENO) == -1) {
        perror("dup2");
        _exit(EXIT_FAILURE);
    }
    if (pipe1[0] != STDIN_FILENO && close_descriptor(pipe1[0]) !=
        ↪ EXIT_SUCCESS) {
        _exit(EXIT_FAILURE);
    }
    if (close_descriptor(pipe1[1]) != EXIT_SUCCESS) {
        _exit(EXIT_FAILURE);
    }
    execv("./child", (char *[]){"./child", filename, NULL});
    perror("execv");
    _exit(EXIT_FAILURE);
}

static int send_commands(int fd) {
    for (;;) {
        char *line = NULL;
        size_t len = 0;
        Status status = read_command_line(&line, &len);
        if (status == STATUS_END_OF_INPUT) {
            return EXIT_SUCCESS;
        }
        if (status != STATUS_OK) {
            fprintf(stderr, "parent: ошибка чтения команды\n");
            return EXIT_FAILURE;
        }
    }
}
```

```

        int result = write_all(fd, line, len);
        if (result == -1) {
            perror("write pipe1");
        }
        free(line);
        if (result == -1) {
            return EXIT_FAILURE;
        }
    }
}

static int wait_for_child(pid_t pid) {
    int status;
    while (waitpid(pid, &status, 0) == -1) {
        if (errno != EINTR) {
            perror("waitpid");
            return EXIT_FAILURE;
        }
    }
    if (!WIFEXITED(status)) {
        fprintf(stderr, "child завершился ненормально\n");
        return EXIT_FAILURE;
    }
    int code = WEXITSTATUS(status);
    if (printf("child завершился с кодом %d\n", code) < 0 || fflush(stdout)
        ↪ == EOF) {
        perror("stdout");
        return EXIT_FAILURE;
    }
    return code == 0 ? EXIT_SUCCESS : EXIT_FAILURE;
}

static int ignore_sigpipe(void) {
    struct sigaction action = {0};
    action.sa_handler = SIG_IGN;
    if (sigemptyset(&action.sa_mask) == -1 || sigaction(SIGPIPE, &action,
        ↪ NULL) == -1) {
        perror("sigaction");
        return EXIT_FAILURE;
    }
    return EXIT_SUCCESS;
}

static int run_parent(int pipe1[2], pid_t pid) {
    int result = close_descriptor(pipe1[0]);
    if (result == EXIT_SUCCESS) {
        result = send_commands(pipe1[1]);
    }
    if (close_descriptor(pipe1[1]) != EXIT_SUCCESS) {
        result = EXIT_FAILURE;
    }
}

```

```

        if (wait_for_child(pid) != EXIT_SUCCESS) {
            result = EXIT_FAILURE;
        }
        return result;
    }

static int run_pipeline(char *filename) {
    if (ignore_sigpipe() != EXIT_SUCCESS) {
        return EXIT_FAILURE;
    }
    int pipe1[2];
    if (pipe(pipe1) == -1) {
        perror("pipe");
        return EXIT_FAILURE;
    }
    if (fflush(stdout) == EOF) {
        perror("fflush");
        close_descriptor(pipe1[0]);
        close_descriptor(pipe1[1]);
        return EXIT_FAILURE;
    }
    pid_t pid = fork();
    if (pid == 0) {
        start_child(pipe1, filename);
    }
    if (pid == -1) {
        perror("fork");
        close_descriptor(pipe1[0]);
        close_descriptor(pipe1[1]);
        return EXIT_FAILURE;
    }
    return run_parent(pipe1, pid);
}

int main(void) {
    char *filename = NULL;
    if (read_filename(&filename) != STATUS_OK) {
        fprintf(stderr, "parent: не удалось прочитать имя файла\n");
        return EXIT_FAILURE;
    }
    int result = run_pipeline(filename);
    free(filename);
    return result;
}

```

Листинг A.2 – Файл parent_input.c

```

#include "parent_input.h"
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>

```

```

static void strip_newline(char *line, ssize_t len) {
    if (len > 0 && line[len - 1] == '\n') {
        line[--len] = '\0';
        if (len > 0 && line[len - 1] == '\r') {
            line[len - 1] = '\0';
        }
    }
}

static ssize_t read_input_line(char **line, size_t *capacity) {
    for (;;) {
        errno = 0;
        ssize_t length = getline(line, capacity, stdin);
        if (length >= 0 || errno != EINTR) {
            return length;
        }
        clearerr(stdin);
    }
}

static Status input_failure_status(void) {
    return feof(stdin) && !ferror(stdin)
        ? STATUS_END_OF_INPUT : STATUS_IO_ERROR;
}

Status read_filename(char **out) {
    *out = NULL;
    char *line = NULL;
    size_t cap = 0;
    ssize_t len = read_input_line(&line, &cap);
    if (len == -1) {
        Status status = input_failure_status();
        free(line);
        return status;
    }
    strip_newline(line, len);
    if (line[0] == '\0') {
        fprintf(stderr, "parent: пустое имя выходного файла\n");
        free(line);
        return STATUS_IO_ERROR;
    }
    *out = line;
    return STATUS_OK;
}

Status read_command_line(char **out, size_t *out_len) {
    *out = NULL;
    *out_len = 0;
    char *line = NULL;
    size_t cap = 0;

```

```

    ssize_t len = read_input_line(&line, &cap);
    if (len == -1) {
        Status status = input_failure_status();
        free(line);
        return status;
    }
    if ((len == 1 && line[0] == '\n') ||
        (len == 2 && line[0] == '\r' && line[1] == '\n')) {
        free(line);
        return STATUS_END_OF_INPUT;
    }
    *out = line;
    *out_len = (size_t)len;
    return STATUS_OK;
}

```

Листинг А.3 – Файл parent_input.h

```

#ifndef PARENT_INPUT_H
#define PARENT_INPUT_H

#include <stddef.h>
#include "status.h"

Status read_filename(char **out);
Status read_command_line(char **out, size_t *out_len);

#endif

```

Листинг А.4 – Файл child.c

```

#include <errno.h>
#include <fcntl.h>
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#include "pipe_io.h"
#include "status.h"
#include "sum.h"

static void report_error(const char *what) {
    fprintf(stderr, "%s: %s\n", what, strerror(errno));
}

static void report(Status status) {
    switch (status) {
        case STATUS_OK:
        case STATUS_END_OF_INPUT:
            break;
    }
}

```

```

        case STATUS_IO_ERROR:
        case STATUS_PIPE_ERROR:
            fprintf(stderr, "child: ошибка ввода-вывода\n");
            break;
        case STATUS_BAD_NUMBER:
        case STATUS_NO_NUMBERS:
            break;
    }
}

static int write_sum(int fd, long sum) {
    char output[sizeof(long) * CHAR_BIT + 2];
    int out_len = snprintf(output, sizeof(output), "%ld\n", sum);
    if (out_len < 0 || (size_t)out_len >= sizeof(output)) {
        fprintf(stderr, "child: не удалось отформатировать сумму\n");
        return EXIT_FAILURE;
    }
    if (write_all(fd, output, (size_t)out_len) == -1) {
        report_error("write");
        return EXIT_FAILURE;
    }
    return EXIT_SUCCESS;
}

static int process_lines(int fd) {
    char *line = NULL;
    size_t cap = 0;
    ssize_t len;
    int exit_status = 0;
    while ((len = read_line(STDIN_FILENO, &line, &cap)) > 0) {
        Status status = STATUS_OK;
        long sum = sum_line(line, &status);
        if (status != STATUS_OK) {
            report(status);
            if (status == STATUS_NO_NUMBERS) {
                continue;
            }
            exit_status = 1;
            if (status != STATUS_BAD_NUMBER) {
                break;
            }
            continue;
        }

        if (write_sum(fd, sum) != EXIT_SUCCESS) {
            exit_status = EXIT_FAILURE;
            break;
        }
    }
    free(line);
    if (len == -1) {

```

```

        report_error("read_line");
        exit_status = 1;
    }
    return exit_status;
}

static int run_child(const char *filename) {
    int fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd == -1) {
        report_error("open");
        return EXIT_FAILURE;
    }
    int exit_status = process_lines(fd);
    if (close(fd) == -1) {
        report_error("close");
        exit_status = 1;
    }
    return exit_status;
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        fprintf(stderr, "usage: %s <output_filename>\n", argv[0]);
        return EXIT_FAILURE;
    }
    if (argv[1][0] == '\\0') {
        fprintf(stderr, "child: пустое имя выходного файла\n");
        return EXIT_FAILURE;
    }
    return run_child(argv[1]);
}

```

Листинг A.5 – Файл sum.c

```

#include "sum.h"
#include <errno.h>
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

long sum_line(const char *line, Status *status) {
    char *copy = strdup(line);
    if (copy == NULL) {
        *status = STATUS_IO_ERROR;
        return 0;
    }

    long sum = 0;
    int had_number = 0;
    *status = STATUS_OK;

```

```

char *saveptr = NULL;
char *tok = strtok_r(copy, " \\t\\r\\n\\v\\f", &saveptr);
while (tok != NULL) {
    char *end = NULL;
    errno = 0;
    long value = strtol(tok, &end, 10);
    if (end == tok || *end != '\\0' || errno == ERANGE ||
        value < INT_MIN || value > INT_MAX) {
        const char *reason = end == tok || *end != '\\0'
            ? "не число" : "число вне диапазона int";
        fprintf(stderr, "child: %s: \"%s\"\\n", reason, tok);
        *status = STATUS_BAD_NUMBER;
    } else if ((value > 0 && sum > LONG_MAX - value) ||
        (value < 0 && sum < LONG_MIN - value)) {
        fprintf(stderr, "child: переполнение суммы\\n");
        *status = STATUS_BAD_NUMBER;
    } else {
        sum += value;
        had_number = 1;
    }
    tok = strtok_r(NULL, " \\t\\r\\n\\v\\f", &saveptr);
}

free(copy);
if (!had_number && *status == STATUS_OK) {
    *status = STATUS_NO_NUMBERS;
}
return sum;
}

```

Листинг А.6 – Файл sum.h

```

#ifndef SUM_H
#define SUM_H

#include "status.h"

long sum_line(const char *line, Status *status);

#endif

```

Листинг А.7 – Файл pipe_io.c

```

#include "pipe_io.h"
#include <unistd.h>
#include <errno.h>
#include <limits.h>
#include <stdint.h>
#include <stdlib.h>

int write_all(int fd, const char *buf, size_t len) {
    size_t written = 0;

```

```

while (written < len) {
    size_t remaining = len - written;
    size_t chunk = remaining > (size_t)SSIZE_MAX ? (size_t)SSIZE_MAX :
        ↪ remaining;
    ssize_t n = write(fd, buf + written, chunk);
    if (n == -1) {
        if (errno == EINTR) {
            continue;
        }
        return -1;
    }
    if (n == 0) {
        errno = EIO;
        return -1;
    }
    written += (size_t)n;
}
return 0;
}

static int ensure_capacity(char **line, size_t *cap, size_t needed) {
    if (needed <= *cap) {
        return 0;
    }
    size_t new_cap = *cap == 0 ? 64 : *cap;
    while (new_cap < needed) {
        if (new_cap > SIZE_MAX / 2) {
            new_cap = needed;
            break;
        }
        new_cap *= 2;
    }
    char *grown = realloc(*line, new_cap);
    if (grown == NULL) {
        return -1;
    }
    *line = grown;
    *cap = new_cap;
    return 0;
}

ssize_t read_line(int fd, char **line, size_t *cap) {
    size_t len = 0;
    size_t consumed = 0;
    for (;;) {
        char c;
        ssize_t n = read(fd, &c, 1);
        if (n == -1) {
            if (errno == EINTR) {
                continue;
            }
        }

```

```

        return -1;
    }
    if (n == 0) {
        break;
    }
    if (consumed == (size_t)SSIZE_MAX) {
        errno = EOVERFLOW;
        return -1;
    }
    consumed++;
    if (c == '\n') {
        break;
    }
    if (c == '\0') {
        errno = EINVAL;
        return -1;
    }
    if (ensure_capacity(line, cap, len + 2) == -1) {
        return -1;
    }
    (*line)[len++] = c;
}
if (ensure_capacity(line, cap, len + 1) == -1) {
    return -1;
}
(*line)[len] = '\0';
return (ssize_t)consumed;
}

```

Листинг A.8 – Файл pipe_io.h

```

#ifndef PIPE_IO_H
#define PIPE_IO_H

#include <stddef.h>
#include <sys/types.h>

int write_all(int fd, const char *buf, size_t len);
ssize_t read_line(int fd, char **line, size_t *cap);

#endif

```

Листинг A.9 – Файл status.h

```

#ifndef STATUS_H
#define STATUS_H

typedef enum {
    STATUS_OK,
    STATUS_END_OF_INPUT,
    STATUS_IO_ERROR,
    STATUS_BAD_NUMBER,

```

```
        STATUS_NO_NUMBERS,  
        STATUS_PIPE_ERROR  
    } Status;  
  
#endif
```

Листинг A.10 – Файл Makefile

```
CC = gcc  
CFLAGS = -std=c11 -Wall -Wextra -Wpedantic -D_POSIX_C_SOURCE=200809L  
  
PARENT_SRCS = parent.c parent_input.c pipe_io.c  
CHILD_SRCS  = child.c sum.c pipe_io.c  
  
all: parent child  
  
parent: $(PARENT_SRCS) status.h parent_input.h pipe_io.h  
        $(CC) $(CFLAGS) -o parent $(PARENT_SRCS)  
  
child: $(CHILD_SRCS) status.h sum.h pipe_io.h  
        $(CC) $(CFLAGS) -o child $(CHILD_SRCS)  
  
clean:  
        rm -f parent child *.o  
  
.PHONY: all clean
```

Приложение Б. Полная трасса лабораторной работы №2

Листинг Б.1 – Вывод strace -f для программы parent

```
1419 execve("./parent", [ "./parent" ], 0x7ffc94c2078 /* 24 vars */) = 0
1419 brk(NULL) = 0x5d8cd8741000
1419 mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
↳ 0x7e3e426a3000
1419 access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
1419 openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
1419 fstat(3, {st_mode=S_IFREG|0644, st_size=35631, ...}) = 0
1419 mmap(NULL, 35631, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7e3e4269a000
1419 close(3) = 0
1419 openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
1419 read(3,
↳ "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"... , 832) =
↳ 832
1419 pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"... ,
↳ 784, 64) = 784
1419 fstat(3, {st_mode=S_IFREG|0755, st_size=2129424, ...}) = 0
1419 pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"... ,
↳ 784, 64) = 784
1419 mmap(NULL, 2174352, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) =
↳ 0x7e3e42400000
1419 mmap(0x7e3e42428000, 1609728, PROT_READ|PROT_EXEC,
↳ MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7e3e42428000
1419 mmap(0x7e3e425b1000, 323584, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
↳ 3, 0x1b1000) = 0x7e3e425b1000
1419 mmap(0x7e3e42600000, 24576, PROT_READ|PROT_WRITE,
↳ MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1ff000) = 0x7e3e42600000
1419 mmap(0x7e3e42606000, 52624, PROT_READ|PROT_WRITE,
↳ MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7e3e42606000
1419 close(3) = 0
1419 mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
↳ 0x7e3e42697000
1419 arch_prctl(ARCH_SET_FS, 0x7e3e42697740) = 0
1419 set_tid_address(0x7e3e42697a10) = 1419
1419 set_robust_list(0x7e3e42697a20, 24) = 0
1419 rseq(0x7e3e42698060, 0x20, 0, 0x53053053) = 0
1419 mprotect(0x7e3e42600000, 16384, PROT_READ) = 0
1419 mprotect(0x5d8ca9cbe000, 4096, PROT_READ) = 0
1419 mprotect(0x7e3e426db000, 8192, PROT_READ) = 0
1419 prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024,
↳ rlim_max=RLIM64_INFINITY}) = 0
1419 munmap(0x7e3e4269a000, 35631) = 0
1419 getrandom("\xa8\x7a\xa5\x1e\xf5\xb8\x81\xde", 8, GRND_NONBLOCK) = 8
1419 brk(NULL) = 0x5d8cd8741000
1419 brk(0x5d8cd8762000) = 0x5d8cd8762000
1419 fstat(0, {st_mode=S_IFREG|0777, st_size=59, ...}) = 0
1419 read(0, "reports/lab2-result.txt\n1 2 3\n10"... , 4096) = 59
1419 rt_sigaction(SIGPIPE, {sa_handler=SIG_IGN, sa_mask=[], sa_flags=SA_RESTORER,
↳ sa_restorer=0x7e3e42445330}, NULL, 8) = 0
1419 pipe2([3, 4], 0) = 0
1419 clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
↳ child_tidptr=0x7e3e42697a10) = 1420
1420 set_robust_list(0x7e3e42697a20, 24 <unfinished ...>
1419 close(3 <unfinished ...>
1420 <... set_robust_list resumed>) = 0
1419 <... close resumed> = 0
```

```

1420 dup2(3, 0 <unfinished ...>
1419 write(4, "1 2 3\n", 6 <unfinished ...>
1420 <... dup2 resumed>) = 0
1419 <... write resumed>) = 6
1420 close(3 <unfinished ...>
1419 write(4, "10 -5\n", 6 <unfinished ...>
1420 <... close resumed>) = 0
1419 <... write resumed>) = 6
1420 close(4 <unfinished ...>
1419 write(4, "-2147483648 2147483647\n", 23 <unfinished ...>
1420 <... close resumed>) = 0
1419 <... write resumed>) = 23
1420 execve("./child", [".child", "reports/lab2-result.txt"], 0x7ffd6497b018 /* 24
↳ vars */ <unfinished ...>
1419 read(0, "", 4096) = 0
1419 close(4) = 0
1419 wait4(1420, <unfinished ...>
1420 <... execve resumed>) = 0
1420 brk(NULL) = 0x627f0fee8000
1420 mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
↳ 0x7526f11ba000
1420 access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
1420 openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
1420 fstat(3, {st_mode=S_IFREG|0644, st_size=35631, ...}) = 0
1420 mmap(NULL, 35631, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7526f11b1000
1420 close(3) = 0
1420 openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
1420 read(3,
↳ "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0\0"... , 832) =
↳ 832
1420 pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"... ,
↳ 784, 64) = 784
1420 fstat(3, {st_mode=S_IFREG|0755, st_size=2129424, ...}) = 0
1420 pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"... ,
↳ 784, 64) = 784
1420 mmap(NULL, 2174352, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) =
↳ 0x7526f0e00000
1420 mmap(0x7526f0e28000, 1609728, PROT_READ|PROT_EXEC,
↳ MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7526f0e28000
1420 mmap(0x7526f0fb1000, 323584, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE,
↳ 3, 0x1b1000) = 0x7526f0fb1000
1420 mmap(0x7526f1000000, 24576, PROT_READ|PROT_WRITE,
↳ MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1ff000) = 0x7526f1000000
1420 mmap(0x7526f1006000, 52624, PROT_READ|PROT_WRITE,
↳ MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7526f1006000
1420 close(3) = 0
1420 mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
↳ 0x7526f11ae000
1420 arch_prctl(ARCH_SET_FS, 0x7526f11ae740) = 0
1420 set_tid_address(0x7526f11aea10) = 1420
1420 set_robust_list(0x7526f11aea20, 24) = 0
1420 rseq(0x7526f11af060, 0x20, 0, 0x53053053) = 0
1420 mprotect(0x7526f1000000, 16384, PROT_READ) = 0
1420 mprotect(0x627ef4a5b000, 4096, PROT_READ) = 0
1420 mprotect(0x7526f11f2000, 8192, PROT_READ) = 0
1420 prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024,
↳ rlim_max=RLIM64_INFINITY}) = 0
1420 munmap(0x7526f11b1000, 35631) = 0

```

```

1420  openat(AT_FDCWD, "reports/lab2-result.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) =
↪ 3
1420  read(0, "1", 1) = 1
1420  getrandom("\x33\xdc\x33\x5e\x80\x5b\x3c\xb8", 8, GRND_NONBLOCK) = 8
1420  brk(NULL) = 0x627f0fee8000
1420  brk(0x627f0ff09000) = 0x627f0ff09000
1420  read(0, " ", 1) = 1
1420  read(0, "2", 1) = 1
1420  read(0, " ", 1) = 1
1420  read(0, "3", 1) = 1
1420  read(0, "\n", 1) = 1
1420  write(3, "6\n", 2) = 2
1420  read(0, "1", 1) = 1
1420  read(0, "0", 1) = 1
1420  read(0, " ", 1) = 1
1420  read(0, "-", 1) = 1
1420  read(0, "5", 1) = 1
1420  read(0, "\n", 1) = 1
1420  write(3, "5\n", 2) = 2
1420  read(0, "-", 1) = 1
1420  read(0, "2", 1) = 1
1420  read(0, "1", 1) = 1
1420  read(0, "4", 1) = 1
1420  read(0, "7", 1) = 1
1420  read(0, "4", 1) = 1
1420  read(0, "8", 1) = 1
1420  read(0, "3", 1) = 1
1420  read(0, "6", 1) = 1
1420  read(0, "4", 1) = 1
1420  read(0, "8", 1) = 1
1420  read(0, " ", 1) = 1
1420  read(0, "2", 1) = 1
1420  read(0, "1", 1) = 1
1420  read(0, "4", 1) = 1
1420  read(0, "7", 1) = 1
1420  read(0, "4", 1) = 1
1420  read(0, "8", 1) = 1
1420  read(0, "3", 1) = 1
1420  read(0, "6", 1) = 1
1420  read(0, "4", 1) = 1
1420  read(0, "7", 1) = 1
1420  read(0, "\n", 1) = 1
1420  write(3, "-1\n", 3) = 3
1420  read(0, "", 1) = 0
1420  close(3) = 0
1420  exit_group(0) = ?
1420  +++ exited with 0 +++
1419  <... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 1420
1419  --- SIGCHLD {si_signo=SIGCHLD, si_code=CLD_EXITED, si_pid=1420, si_uid=1000,
↪ si_status=0, si_utime=0, si_stime=0} ---
1419  fstat(1, {st_mode=S_IFREG|0777, st_size=0, ...}) = 0
1419  write(1, "child
↪ \320\267\320\260\320\262\320\265\321\200\321\210\320\270\320\273\321\201\321\217
↪ \321\201 \320\272"... , 43) = 43
1419  exit_group(0) = ?
1419  +++ exited with 0 +++

```

Приложение В. Полная трасса GNU ls

Листинг В.1 – Вывод strace для GNU ls

```
execve("/usr/bin/ls", ["ls", "-l", "report/inc/fixture"], 0x7ffcc0594a90 /* 25 vars
↳ */) = 0
brk(NULL) = 0x61801edc8000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
↳ 0x73d60711f000
access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=35631, ...}) = 0
mmap(NULL, 35631, PROT_READ, MAP_PRIVATE, 3, 0) = 0x73d607116000
close(3) = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libselinux.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0"..., 832) =
↳ 832
fstat(3, {st_mode=S_IFREG|0644, st_size=174472, ...}) = 0
mmap(NULL, 181960, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x73d6070e9000
mmap(0x73d6070ef000, 118784, PROT_READ|PROT_EXEC,
↳ MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x6000) = 0x73d6070ef000
mmap(0x73d60710c000, 24576, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
↳ 0x23000) = 0x73d60710c000
mmap(0x73d607112000, 8192, PROT_READ|PROT_WRITE,
↳ MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x29000) = 0x73d607112000
mmap(0x73d607114000, 5832, PROT_READ|PROT_WRITE,
↳ MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x73d607114000
close(3) = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0"...,
↳ 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0"..., 784,
↳ 64) = 784
fstat(3, {st_mode=S_IFREG|0755, st_size=2129424, ...}) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0"..., 784,
↳ 64) = 784
mmap(NULL, 2174352, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x73d606e00000
mmap(0x73d606e28000, 1609728, PROT_READ|PROT_EXEC,
↳ MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x73d606e28000
mmap(0x73d606fb1000, 323584, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
↳ 0x1b1000) = 0x73d606fb1000
mmap(0x73d607000000, 24576, PROT_READ|PROT_WRITE,
↳ MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1ff000) = 0x73d607000000
mmap(0x73d607006000, 52624, PROT_READ|PROT_WRITE,
↳ MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x73d607006000
close(3) = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libpcre2-8.so.0", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0"..., 832) =
↳ 832
fstat(3, {st_mode=S_IFREG|0644, st_size=625344, ...}) = 0
mmap(NULL, 627472, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x73d60704f000
mmap(0x73d607051000, 450560, PROT_READ|PROT_EXEC,
↳ MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x2000) = 0x73d607051000
mmap(0x73d6070bf000, 163840, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
↳ 0x70000) = 0x73d6070bf000
mmap(0x73d6070e7000, 8192, PROT_READ|PROT_WRITE,
↳ MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x97000) = 0x73d6070e7000
close(3) = 0
```

```

mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
↳ 0x73d60704c000
arch_prctl(ARCH_SET_FS, 0x73d60704c800) = 0
set_tid_address(0x73d60704cad0) = 1414
set_robust_list(0x73d60704cae0, 24) = 0
rseq(0x73d60704d120, 0x20, 0, 0x53053053) = 0
mprotect(0x73d607000000, 16384, PROT_READ) = 0
mprotect(0x73d6070e7000, 4096, PROT_READ) = 0
mprotect(0x73d607112000, 4096, PROT_READ) = 0
mprotect(0x61800db5e000, 8192, PROT_READ) = 0
mprotect(0x73d607157000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
munmap(0x73d607116000, 35631) = 0
statfs("/sys/fs/selinux", {f_type=SYSFS_MAGIC, f_bsize=4096, f_blocks=0, f_bfree=0,
↳ f_bavail=0, f_files=0, f_ffree=0, f_fsid={val=[0xe555e996, 0xf5f48777]},
↳ f_namelen=255, f_frsize=4096,
↳ f_flags=ST_VALID|ST_NOSUID|ST_NODEV|ST_NOEXEC|ST_NOATIME}) = 0
statfs("/selinux", 0x7ffea8b661d0) = -1 ENOENT (No such file or directory)
getrandom("\xcd\x58\xed\xff\x5c\x2a\x90\xb1", 8, GRND_NONBLOCK) = 8
brk(NULL) = 0x61801edc8000
brk(0x61801ede9000) = 0x61801ede9000
openat(AT_FDCWD, "/proc/filesystems", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0444, st_size=0, ...}) = 0
read(3, "nodev\tsysfs\nnodev\ttmpfs\nnodev\tbd"... , 1024) = 442
close(3) = 0
openat(AT_FDCWD, "/proc/mounts", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0444, st_size=0, ...}) = 0
read(3, "/dev/sde / ext4 rw,relatime,disc"... , 1024) = 1024
read(3, " tmpfs rw,nosuid,nodev,mode=755 "... , 1024) = 1024
read(3, "/6.6.87.2-microsoft-standard-WSL"... , 1024) = 606
read(3, "", 1024) = 0
close(3) = 0
access("/etc/selinux/config", F_OK) = -1 ENOENT (No such file or directory)
ioctl(1, TCGETS, 0x7ffea8b66070) = -1 ENOTTY (Inappropriate ioctl for device)
statx(AT_FDCWD, "report/inc/fixture",
↳ AT_STATX_SYNC_AS_STAT|AT_SYMLINK_NOFOLLOW|AT_NO_AUTOMOUNT,
↳ STATX_MODE|STATX_NLINK|STATX_UID|STATX_GID|STATX_MTIME|STATX_SIZE,
↳ {stx_mask=STATX_BASIC_STATS|STATX_MNT_ID, stx_attributes=0,
↳ stx_mode=S_IFDIR|0777, stx_size=4096, ...}) = 0
lgetxattr("report/inc/fixture", "security.selinux", 0x61801edcddb0, 255) = -1
↳ ENODATA (No data available)
listxattr("report/inc/fixture", "", 152) = 0
socket(AF_UNIX, SOCK_STREAM|SOCK_CLOEXEC|SOCK_NONBLOCK, 0) = 3
connect(3, {sa_family=AF_UNIX, sun_path="/var/run/nscd/socket"}, 110) = -1 ENOENT
↳ (No such file or directory)
close(3) = 0
socket(AF_UNIX, SOCK_STREAM|SOCK_CLOEXEC|SOCK_NONBLOCK, 0) = 3
connect(3, {sa_family=AF_UNIX, sun_path="/var/run/nscd/socket"}, 110) = -1 ENOENT
↳ (No such file or directory)
close(3) = 0
newfstatat(AT_FDCWD, "/etc/nsswitch.conf", {st_mode=S_IFREG|0644, st_size=526, ...},
↳ 0) = 0
newfstatat(AT_FDCWD, "/", {st_mode=S_IFDIR|0755, st_size=4096, ...}, 0) = 0
openat(AT_FDCWD, "/etc/nsswitch.conf", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=526, ...}) = 0
read(3, "# /etc/nsswitch.conf\n#\n# Example"... , 4096) = 526
read(3, "", 4096) = 0
fstat(3, {st_mode=S_IFREG|0644, st_size=526, ...}) = 0

```

